```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv(r"/content/Social_Network_Ads.csv")
```

```
df
```

	Age	EstimatedSalary	Purchased
0	19	19000	0
1	35	20000	0
2	26	43000	0
3	27	57000	0
4	19	76000	0
...	...	...	...
395	46	41000	1
396	51	23000	1
397	50	20000	1
398	36	33000	0
399	49	36000	1

400 rows × 3 columns

Next steps: Generate code with df | New interactive sheet

Train test split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df.drop('Purchased', axis=1
                                                    df['Purchased'],
                                                    test_size=0.3,
                                                    random_state=0)


X_train.shape, X_test.shape
```

```
((280, 2), (120, 2))
```

StandardScaler

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
scaler.mean_
```

```
array([3.78642857e+01, 6.98071429e+04])
```

```
X_train
```

	Age	EstimatedSalary
92	26	15000
223	60	102000
234	38	112000
232	40	107000
377	42	53000
...	...	...
323	48	30000
192	29	43000
117	36	52000
47	27	54000
172	26	118000

280 rows × 2 columns

Next steps: Generate code with X_train | New interactive sheet

```
X_train_scaled
```

```
[ 0.11134522,  1.8563962 ],
[-0.86905295, -0.775237  ],
[-0.47689368, -0.775237  ],
[-0.28081405, -0.91983223],
[ 0.30742485, -0.71739891],
[ 0.30742485,  0.06341534],
[ 0.11134522,  1.8563962 ],
[-1.06513258,  1.94315334],
[-1.65337148, -1.55605125],
[-1.1631724 , -1.09334651],
[-0.67297331, -0.11009894],
[ 0.11134522,  0.09233438],
[ 0.30742485,  0.26584866],
[ 0.89566375, -0.57280368],
[ 0.30742485, -1.1511846 ],
[-0.08473441,  0.67071531],
[ 2.17018137, -0.68847986],
[-1.26121221, -1.38253697],
[-0.96709276, -0.94875128],
[ 0.0133054 , -0.42820845],
[-0.18277423, -0.45712749],
[-1.7514113 , -0.97767033],
[ 1.7780221 ,  0.98882482],
[ 0.20938504, -0.37037036],
[ 0.40546467,  1.104501  ],
[-1.7514113 , -1.35361793],
[ 0.20938504, -0.13901799],
[ 0.89566375, -1.44037507],
[-1.94749093,  0.46828198],
[-0.28081405,  0.26584866],
[ 1.87606192, -1.06442747],
[-0.37885386,  0.06341534],
[ 1.09174339, -0.89091319],
[-1.06513258, -1.12226556],
[-1.84945111,  0.00557724],
[ 0.11134522,  0.26584866],
[-1.1631724 ,  0.32368675],
[-1.26121221,  0.29476771],
[-0.96709276,  0.43936294],
[ 1.67998229, -0.89091319],
[ 1.1897832 ,  0.52612008],
[ 1.09174339,  0.52612008],
[ 1.38586284,  2.31910094],
[-0.28081405, -0.13901799],
[ 0.40546467, -0.45712749],
[-0.37885386, -0.775237  ],
[-0.08473441, -0.51496559],
[ 0.99370357, -1.1511846 ],
[-0.86905295, -0.775237  ],
[-0.18277423, -0.51496559],
[-1.06513258, -0.45712749],
[-1.1631724 ,  1.39369146]])
```

```
X_train_scaled = pd.DataFrame(X_train_scaled, columns=df.drop('Purchased', axis
X_test_scaled = pd.DataFrame(X_test_scaled, columns=df.drop('Purchased', axis=1
```

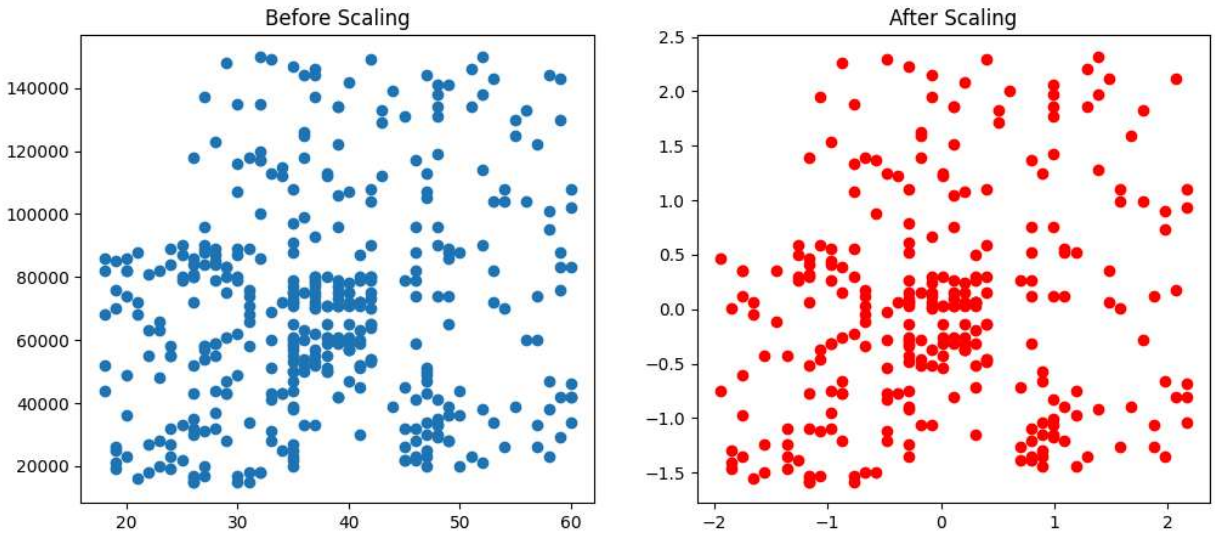
```
X_train_scaled.describe()
```

	Age	EstimatedSalary
count	2.800000e+02	2.800000e+02
mean	3.489272e-17	6.344132e-17
std	1.001791e+00	1.001791e+00
min	-1.947491e+00	-1.584970e+00
25%	-7.710131e-01	-7.752370e-01
50%	-8.473441e-02	2.003677e-02
75%	7.976239e-01	5.261201e-01
max	2.170181e+00	2.319101e+00

```
np.round(X_train_scaled.describe(), 1)
```

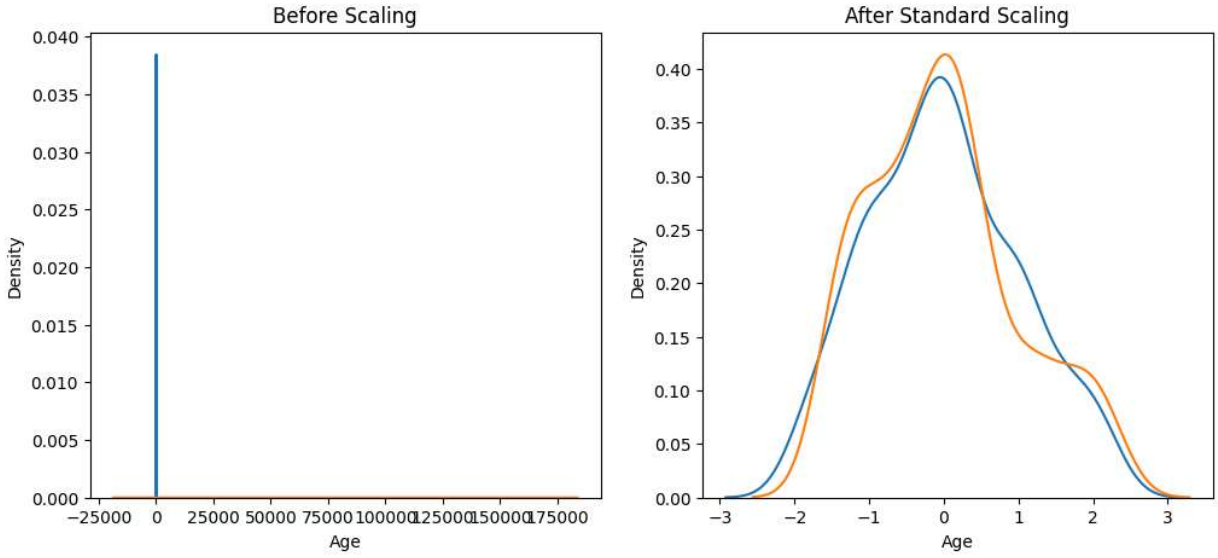
	Age	EstimatedSalary
count	280.0	280.0
mean	0.0	0.0
std	1.0	1.0
min	-1.9	-1.6
25%	-0.8	-0.8
50%	-0.1	0.0
75%	0.8	0.5
max	2.2	2.3

```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12, 5))

ax1.scatter(df['Age'], df['EstimatedSalary'])
ax1.set_title("Before Scaling")
ax2.scatter(X_train_scaled['Age'], X_train_scaled['EstimatedSalary'],color='rec
ax2.set_title("After Scaling")
plt.show()
```



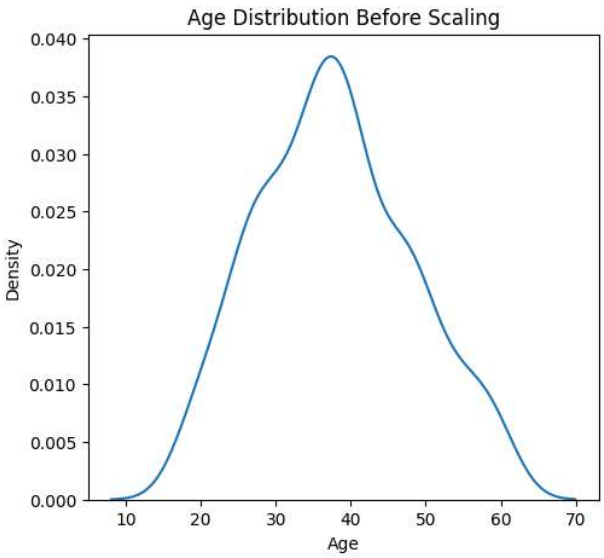

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Re-defining df, X_train, and X_train_scaled for independent cell execution
df = pd.read_csv(r"/content/Social_Network_Ads.csv")
X_train, X_test, y_train, y_test = train_test_split(df.drop('Purchased', axis=1
                                                    df['Purchased'],
                                                    test_size=0.3,
                                                    random_state=0)

scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
X_train_scaled = pd.DataFrame(X_train_scaled, columns=df.drop('Purchased', axis
X_test_scaled = pd.DataFrame(X_test_scaled, columns=df.drop('Purchased', axis=1

fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12,5 ))

# before scaling
ax1.set_title('Before Scaling')
sns.kdeplot(X_train['Age'], ax=ax1)
sns.kdeplot(X_train['EstimatedSalary'], ax=ax1)
```

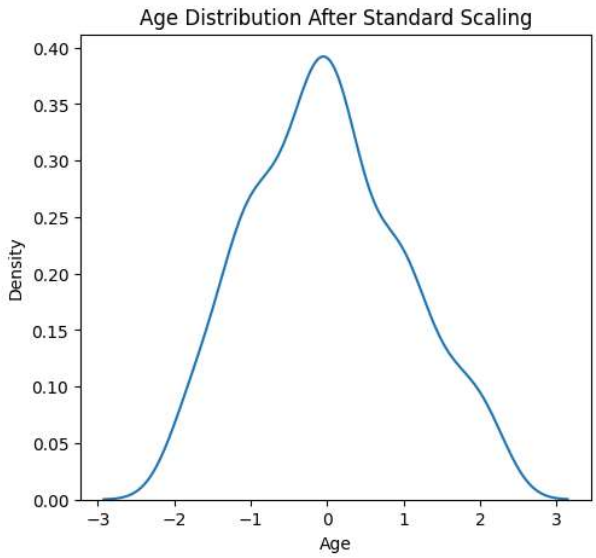
```
# after scaling
ax2.set_title('After Standard Scaling')
sns.kdeplot(X_train_scaled['Age'], ax=ax2)
sns.kdeplot(X_train_scaled['EstimatedSalary'], ax=ax2)
plt.show()
```



COMPARISON OF DISTRIBUTION

```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12,5))

# before scaling
ax1.set_title('Age Distribution Before Scaling')
sns.kdeplot(X_train['Age'], ax=ax1)

# after scaling
ax2.set_title('Age Distribution After Standard Scaling')
sns.kdeplot(X_train_scaled['Age'], ax=ax2)
plt.show()
```





WHY SCALING IS IMPORTANT

```
from sklearn.linear_model import LogisticRegression
```

```
lr = LogisticRegression()
lr_scaled = LogisticRegression()
```

```
lr.fit(X_train, y_train)
lr_scaled.fit(X_train_scaled, y_train)
```

```
LogisticRegression()
```

```
y_pred = lr.predict(X_test)
y_pred_scaled = lr_scaled.predict(X_test_scaled)
```

```
from sklearn.metrics import accuracy_score
```

```
print("Actual",accuracy_score(y_test, y_pred))
print("Scaled",accuracy_score(y_test, y_pred_scaled))
```

```
Actual 0.875
Scaled 0.8666666666666667
```

```
df.describe()
```

	Age	EstimatedSalary	Purchased
count	400.000000	400.000000	400.000000
mean	37.655000	69742.500000	0.357500
std	10.482877	34096.960282	0.479864
min	18.000000	15000.000000	0.000000
25%	29.750000	43000.000000	0.000000
50%	37.000000	70000.000000	0.000000
75%	46.000000	88000.000000	1.000000
max	60.000000	150000.000000	1.000000

```
df = pd.concat([df, pd.DataFrame({'Age':[5,90,95],'EstimatedSalary':[1000,25000
```

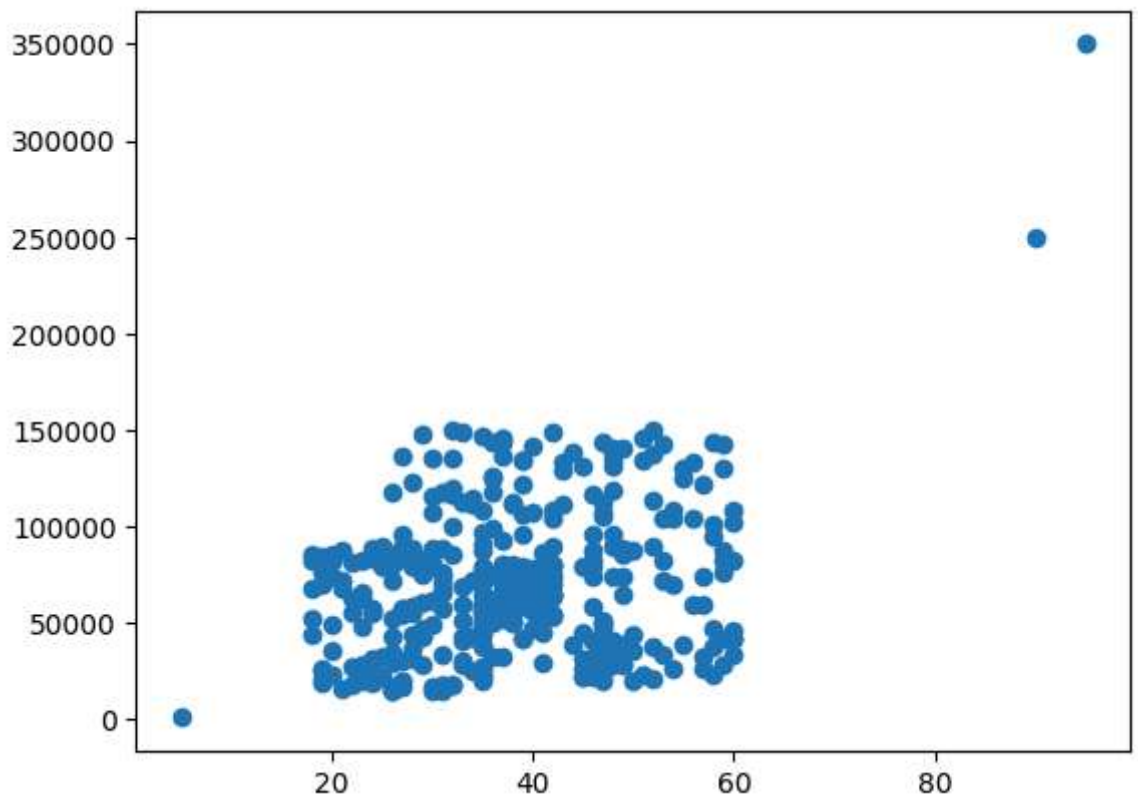
```
df
```

	Age	EstimatedSalary	Purchased
0	19	19000	0
1	35	20000	0
2	26	43000	0
3	27	57000	0
4	19	76000	0
...	...	...	...
398	36	33000	0
399	49	36000	1
400	5	1000	0
401	90	250000	1
402	95	350000	1

403 rows × 3 columns

Next steps: Generate code with df | New interactive sheet

```
plt.scatter(df['Age'], df['EstimatedSalary'])
plt.show()
```



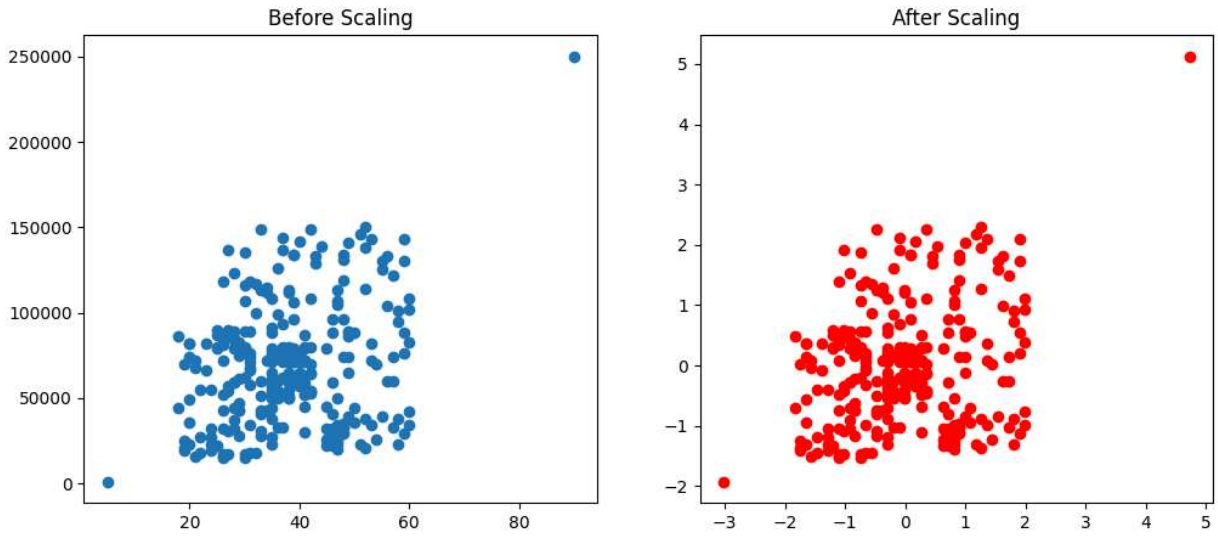

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df.drop('Purchased', axis=1
                                                    df['Purchased'],
                                                    test_size=0.3,
                                                    random_state=0)

X_train.shape, X_test.shape
```

```
((282, 2), (121, 2))
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
X_train_scaled = pd.DataFrame(X_train_scaled, columns=df.drop('Purchased', axis
X_test_scaled = pd.DataFrame(X_test_scaled, columns=df.drop('Purchased', axis=1
```

```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12,5))

ax1.scatter(X_train['Age'], X_train['EstimatedSalary'])
ax1.set_title("Before Scaling")
ax2.scatter(X_train_scaled['Age'], X_train_scaled['EstimatedSalary'],color='rec
ax2.set_title("After Scaling")
plt.show()
```



#NORMALIZATION

```
df = pd.read_csv(r"/content/wine_data.csv")
```

```
df
```

	class_label	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_
0	1	14.23	1.71	2.43	15.6	127	
1	1	13.20	1.78	2.14	11.2	100	
2	1	13.16	2.36	2.67	18.6	101	
3	1	14.37	1.95	2.50	16.8	113	
4	1	13.24	2.59	2.87	21.0	118	
...	...	...	...	...	...	...	
173	3	13.71	5.65	2.45	20.5	95	
174	3	13.40	3.91	2.48	23.0	102	
175	3	13.27	4.28	2.26	20.0	120	
176	3	13.17	2.59	2.37	20.0	120	
177	3	14.13	4.10	2.74	24.5	96	

178 rows × 14 columns

Next steps: Generate code with df | New interactive sheet

```
df = pd.read_csv('wine_data.csv',header=None,usecols=[0,1,2])
df.columns=['Class label','Alcohol','Malic acid']
```

```
df
```

	Class label	Alcohol	Malic acid
0	class_label	alcohol	malic_acid
1	1	14.23	1.71
2	1	13.2	1.78
3	1	13.16	2.36
4	1	14.37	1.95
...	...	...	...
174	3	13.71	5.65
175	3	13.4	3.91
176	3	13.27	4.28
177	3	13.17	2.59
178	3	14.13	4.1

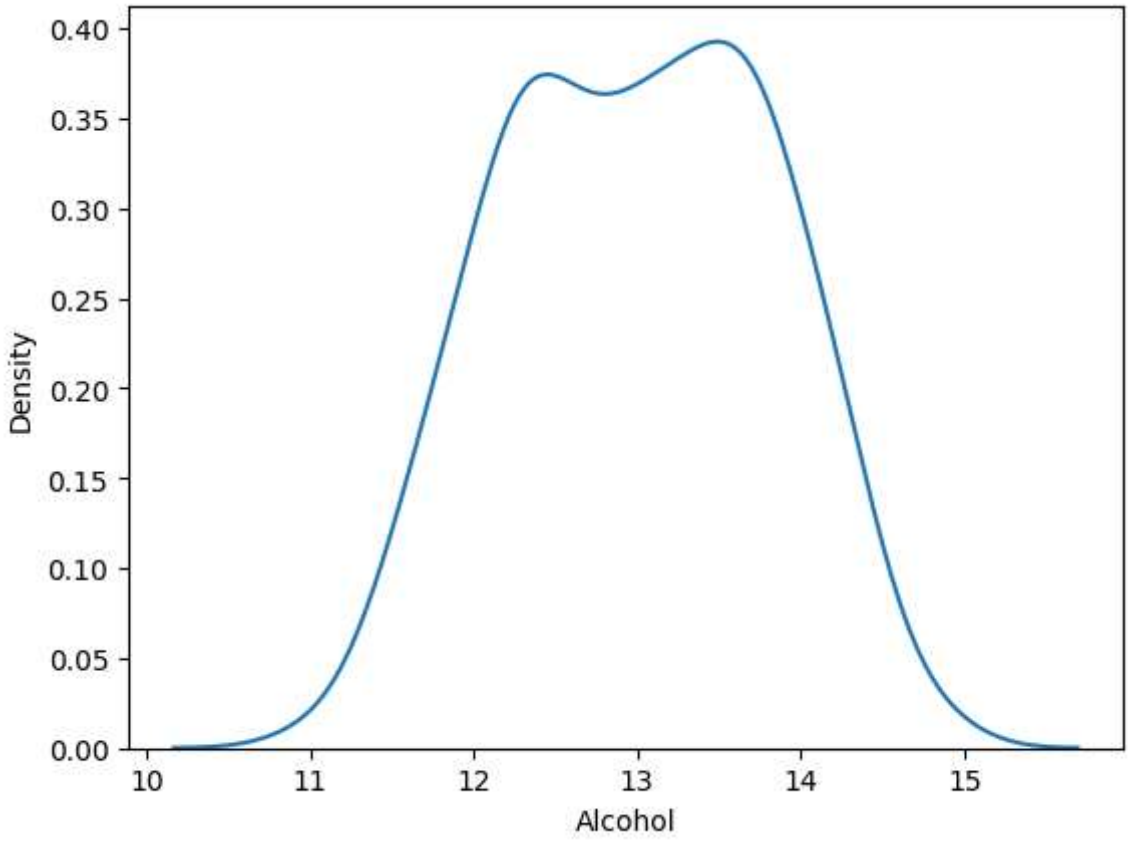
179 rows × 3 columns

Next steps: Generate code with df | New interactive sheet

```
import seaborn as sns
```

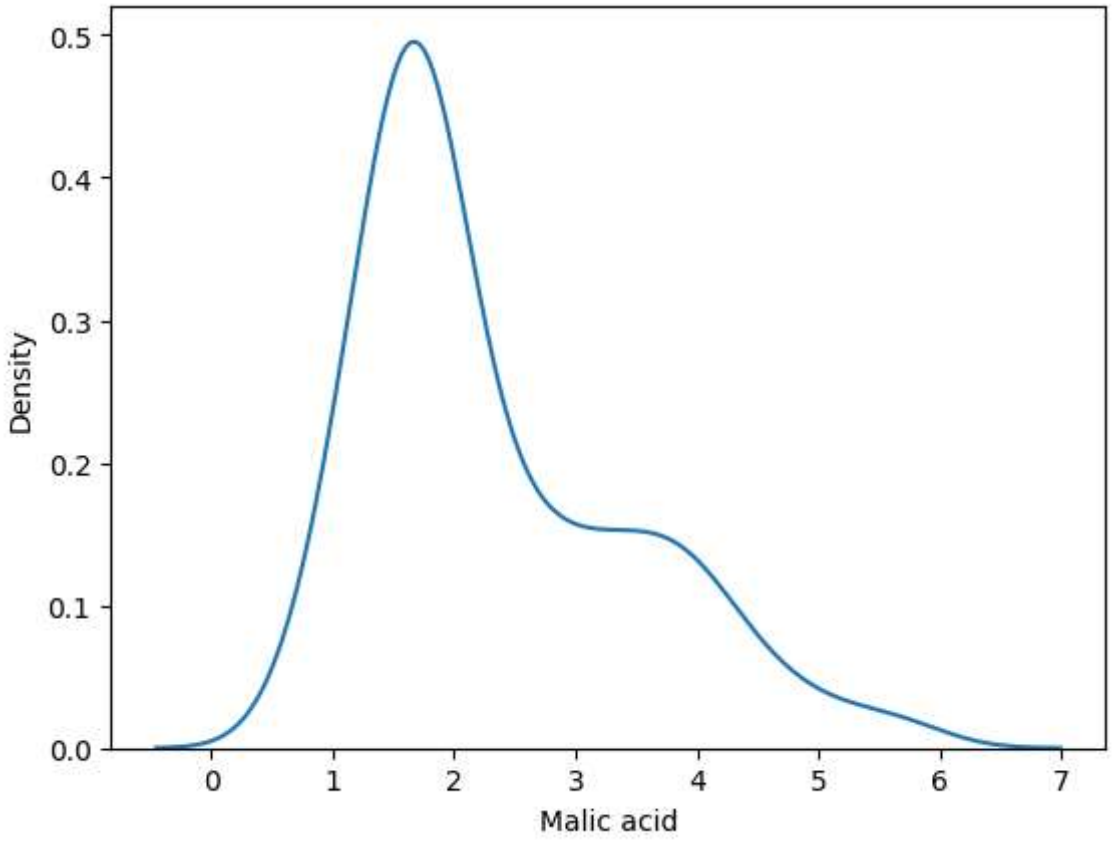
```
import pandas as pd

# Convert the 'Alcohol' column to a numeric type, coercing errors to NaN.
# This will turn the string 'alcohol' into a NaN value.
numeric_alcohol = pd.to_numeric(df['Alcohol'], errors='coerce')

# Drop the NaN values (which corresponds to the original header row string 'alc
# before plotting, as kdeplot expects purely numeric data.
sns.kdeplot(numeric_alcohol.dropna())
```

```
<Axes: xlabel='Alcohol', ylabel='Density'>
```




```
import pandas as pd

numeric_malic_acid = pd.to_numeric(df['Malic acid'], errors='coerce')
sns.kdeplot(numeric_malic_acid.dropna())
```

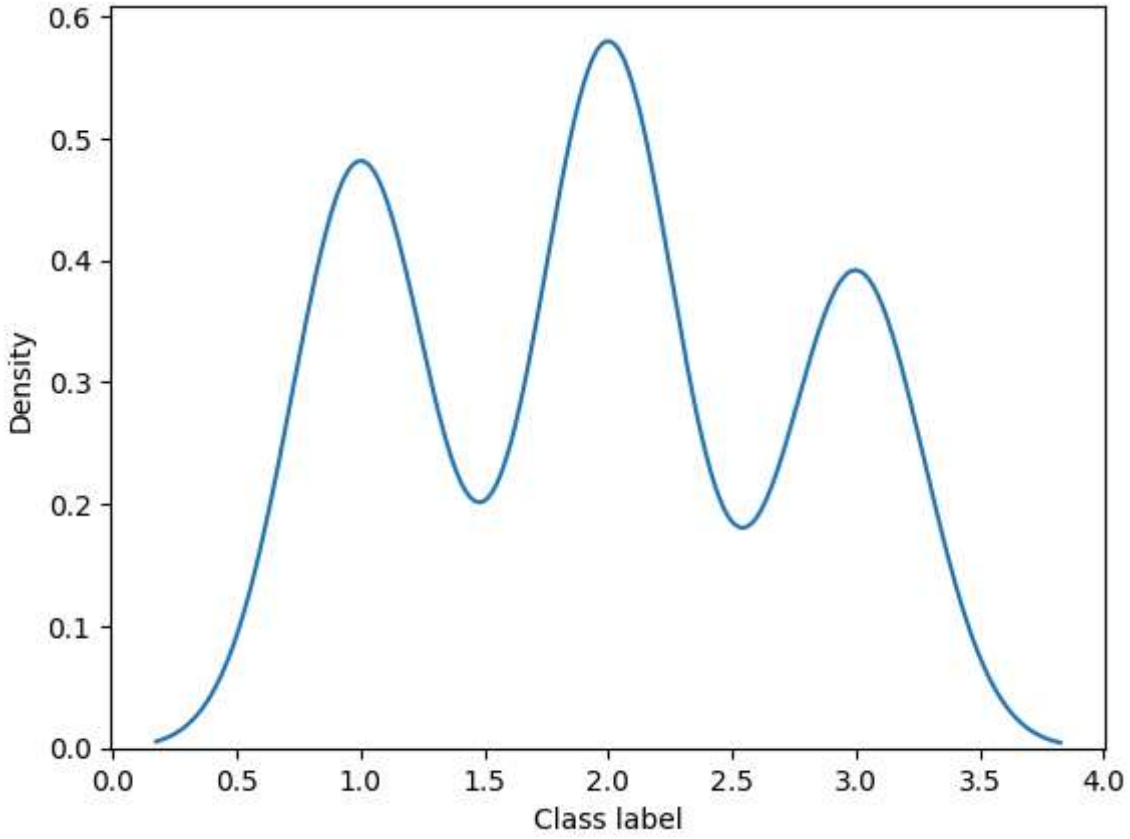
```
<Axes: xlabel='Malic acid', ylabel='Density'>
```




```
Start coding or generate with AI.
```

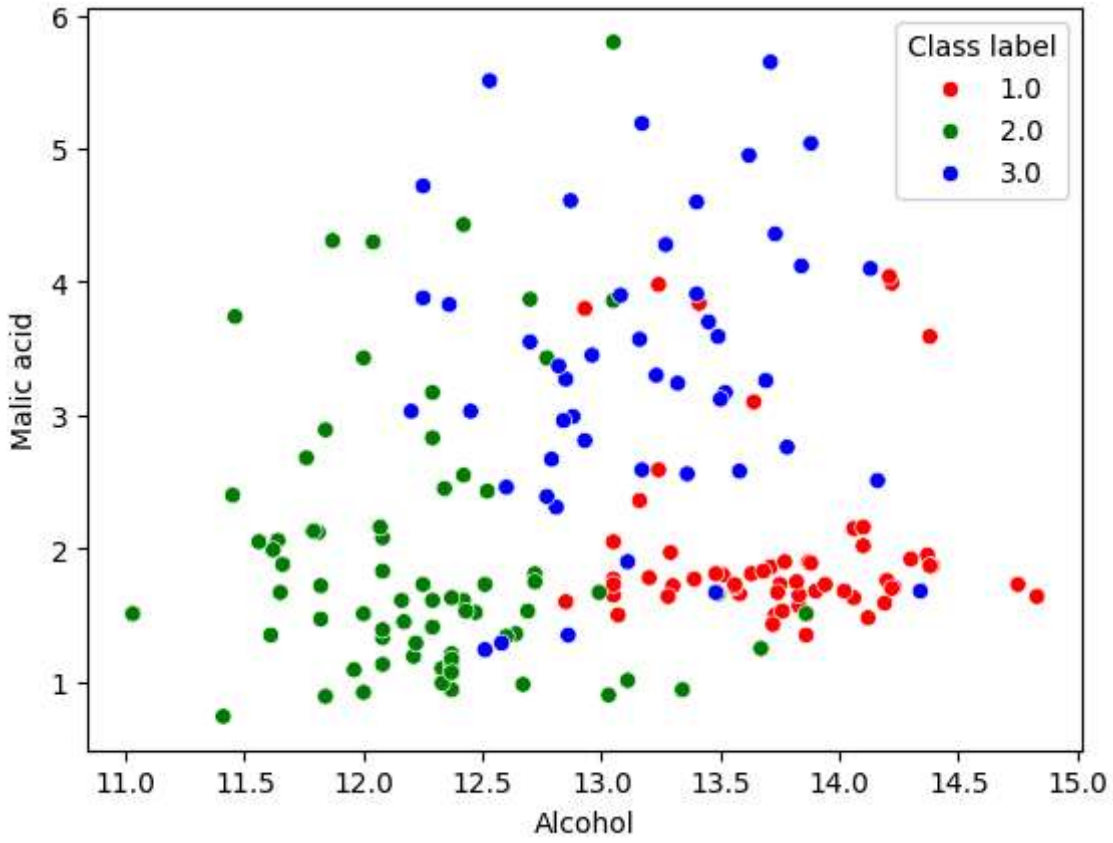
```
import pandas as pd

numeric_class_label = pd.to_numeric(df['Class label'], errors='coerce')
sns.kdeplot(numeric_class_label.dropna())
```

```
<Axes: xlabel='Class label', ylabel='Density'>
```




```
import pandas as pd

color_dict={1:'red',2:'green',3:'blue'}

# Convert columns to numeric, coercing errors, and drop NaNs
numeric_alcohol = pd.to_numeric(df['Alcohol'], errors='coerce').dropna()
numeric_malic_acid = pd.to_numeric(df['Malic acid'], errors='coerce').dropna()
numeric_class_label = pd.to_numeric(df['Class label'], errors='coerce').dropna(

# Align indices for plotting, considering that NaNs were dropped
# We'll create a temporary DataFrame for plotting to ensure indices match
plot_df = pd.DataFrame({
    'Alcohol': numeric_alcohol,
    'Malic acid': numeric_malic_acid,
    'Class label': numeric_class_label
}).dropna()

sns.scatterplot(x=plot_df['Alcohol'], y=plot_df['Malic acid'], hue=plot_df['Cla
```

```
<Axes: xlabel='Alcohol', ylabel='Malic acid'>
```




```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df.drop('Class label', axis
                                                    df['Class label'],
                                                    test_size=0.3,
                                                    random_state=0)

X_train.shape, X_test.shape
```

```
((125, 2), (54, 2))
```

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd # Import pandas for data manipulation

scaler = MinMaxScaler()

# Convert columns in X_train and X_test to numeric, coercing errors to NaN
X_train_numeric = X_train.apply(pd.to_numeric, errors='coerce')
X_test_numeric = X_test.apply(pd.to_numeric, errors='coerce')

# Identify rows with NaN values (which were originally strings) in X_train and
nan_rows_train = X_train_numeric.isnull().any(axis=1)
nan_rows_test = X_test_numeric.isnull().any(axis=1)

# Filter X_train and y_train to remove rows with non-numeric data
X_train = X_train_numeric[~nan_rows_train]
```

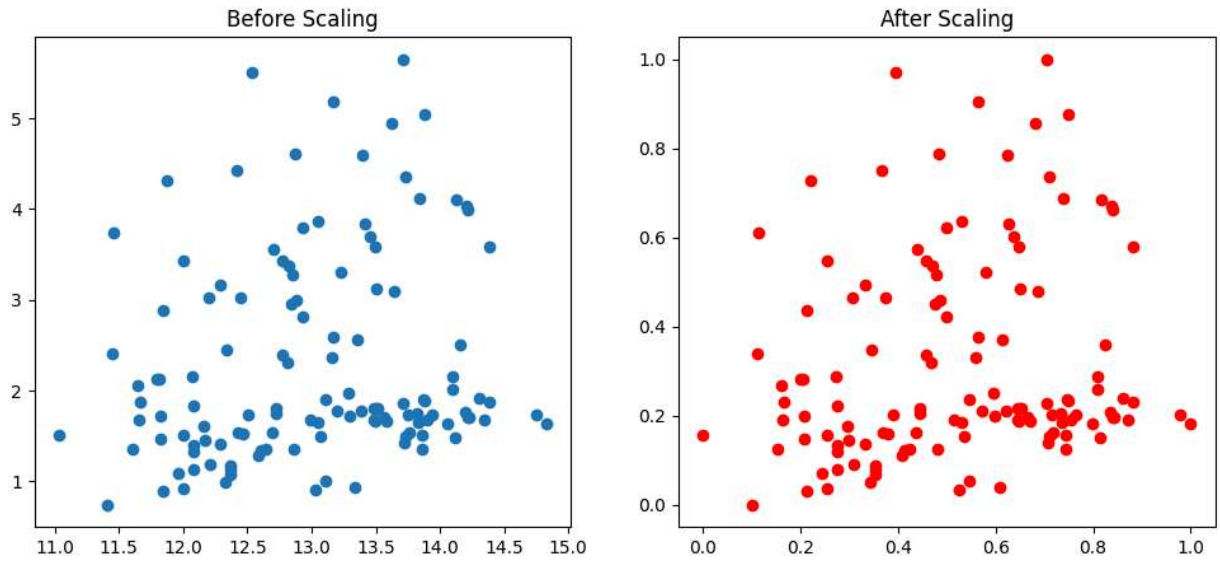
```
y_train = y_train[~nan_rows_train]

# Filter X_test and y_test to remove rows with non-numeric data
X_test = X_test_numeric[~nan_rows_test]
y_test = y_test[~nan_rows_test]

# fit the scaler to the train set, it will learn the parameters
scaler.fit(X_train)

# transform train and test sets
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
X_train_scaled = pd.DataFrame(X_train_scaled, columns=df.drop('Class label', ax
X_test_scaled = pd.DataFrame(X_test_scaled, columns=df.drop('Class label', axis
```

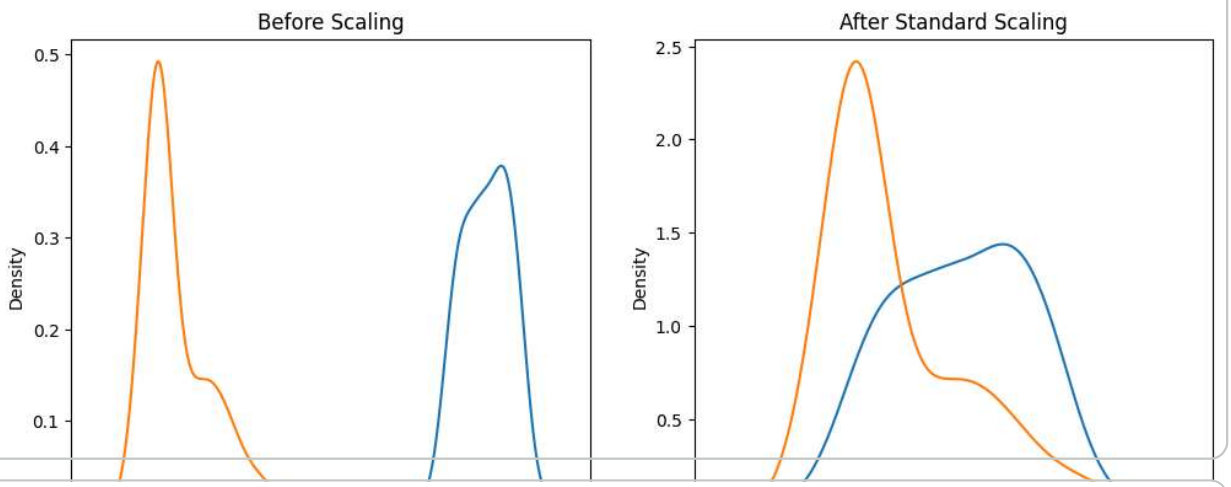
```
np.round(X_train.describe(), 1)
```

	Alcohol	Malic acid
count	124.0	124.0
mean	13.0	2.3
std	0.9	1.1
min	11.0	0.7
25%	12.3	1.5
50%	13.0	1.8
75%	13.7	3.0
max	14.8	5.6

```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12,5))

ax1.scatter(X_train['Alcohol'], X_train['Malic acid'])
ax1.set_title("Before Scaling")
ax2.scatter(X_train_scaled['Alcohol'], X_train_scaled['Malic acid'],color='red'
ax2.set_title("After Scaling")
plt.show()
```




```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12,5))

# before scaling
ax1.set_title('Before Scaling')
sns.kdeplot(X_train['Alcohol'], ax=ax1)
sns.kdeplot(X_train['Malic acid'], ax=ax1)

# after scaling
ax2.set_title('After Standard Scaling')
sns.kdeplot(X_train_scaled['Alcohol'], ax=ax2)
sns.kdeplot(X_train_scaled['Malic acid'], ax=ax2)
plt.show()
```




```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12,5))

# before scaling
```